

Common Problems



Design a News Aggregator

Scaling Reads

By Evan King · Published Jun 2, 2025 · easy · Asked at:

Understanding the Problem

What is Google News?

Google News is a digital service that aggregates and displays news articles from thousands of publishers worldwide in a scrollable interface for users to stay updated on current events.

Functional Requirements

Core Requirements

1. Users should be able to view an aggregated feed of news articles from thousands of source publishers all over the world
2. Users should be able to scroll through the feed "infinitely"
3. Users should be able to click on articles and be redirected to the publisher's website to read the full content

Below the line (out of scope):

- Users should be able to customize their feed based on interests
- Users should be able to save articles for later reading
- Users should be able to share articles on social media platforms

Non-Functional Requirements



For a news platform, availability is prioritized over consistency, as users would prefer to see slightly outdated content rather than no content at all.

Core Requirements



1. The system should prioritize availability over consistency (CAP theorem)
2. The system should be scalable to handle 100 million daily active users with spikes up to 500 million
3. The system should have low latency feed load times (< 200ms)

Below the line (out of scope):

- The system should protect user data and privacy
- The system should handle traffic spikes during breaking news events

- The system should have appropriate monitoring and observability
- The system should be resilient against publisher API failures Here's how it might look on your whiteboard:

Functional Requirements

- view news from publishers around the world
- infinite scroll
- redirect to article on publisher website

Non-functional Requirements

- availability >> consistency
- scale to spike of 500m DAU
- low latency feed load (<200ms)



IG Requirements

The Set Up

Planning the Approach

Before diving into the design, I'll follow the framework by building sequentially through our functional requirements and using non-functional requirements to guide our deep dives. For Google News, we'll need to carefully balance scalability and performance to meet our high traffic demands.

Defining the Core Entities

I like to begin with a broad overview of the primary entities we'll need. At this stage, it's not necessary to know every specific column or detail - we'll focus on those intricacies later when we have a clearer grasp. Initially, establishing these key entities will guide our thought process and lay a solid foundation.



When communicating your entity design, focus on explaining the relationships between entities and their purpose rather than listing every attribute.

To satisfy our key functional requirements, we'll need the following entities:

1. **Article:** Represents a news article with attributes like id, title, summary, thumbnail URL, publish date, publisher ID, region, and media URLs. This is our core content entity.
2. **Publisher:** Represents a news source with attributes like id, name, URL, feed URL, and region. Publishers are the origin of our content.
3. **User:** Represents system users with attributes like id and region (which may be inferred from IP or explicitly set). Even with anonymous users, we track basic information.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.

Up to 15% off select mock interviews 🎉



Ask me anything about this topic

Core Entities

- Article
- Publisher
- User



News Feed Entities

API or System Interface

The API is the main way users will interact with our news feed system. Defining it early helps us structure the rest of our design. I'll create endpoints for each of our core requirements.

For users to view an aggregated feed of news articles:

```
// Get a page of articles for the user's feed  
GET /feed?page={page}&limit={limit}&region={region} -> Article[]
```



We're starting with simple offset-based pagination for now, but this has performance issues for infinite scrolling. We'll improve this to cursor-based pagination in our deep dive to handle the scale and user experience requirements better.

For users to view a specific article we don't need an API endpoint, since their browser will navigate to the publisher's website once they click on the article based on the `url` field in the article object.

High-Level Design

We'll build our design progressively, addressing each functional requirement one by one and adding the necessary components as we go. For Google News, we need to handle both the ingestion of content from thousands of publishers and the efficient delivery of that content to millions of users.

1) Users should be able to view an aggregated feed of news articles from thousands of source publishers all over the world

Users need to see a personalized feed of recent news articles when they visit Google News. This involves two distinct challenges: collecting content from publishers and serving it to users efficiently.

We'll start with collecting data from publishers. To do this, we need a **Data Collection Service** that runs as a background process to continuously gather content from thousands of news sources:

1. **Data Collection Service:** Polls publisher RSS feeds and APIs every 3-6 hours based on each publisher's update frequency.
2. **Publishers:** Thousands of news sources worldwide that provide content via RSS feeds or APIs
3. **Database:** Stores collected articles, publishers, and metadata

4. **Object Storage:** Stores thumbnails for the articles

Our Data Collection Service workflow:

1. Data Collection Service queries the database for the list of publishers and their RSS feed URLs before querying each one after another.
2. Extracts article content, metadata, and downloads media files to use as thumbnails.
3. Stores thumbnail files in Object Storage and saves article data with media URLs to the Database



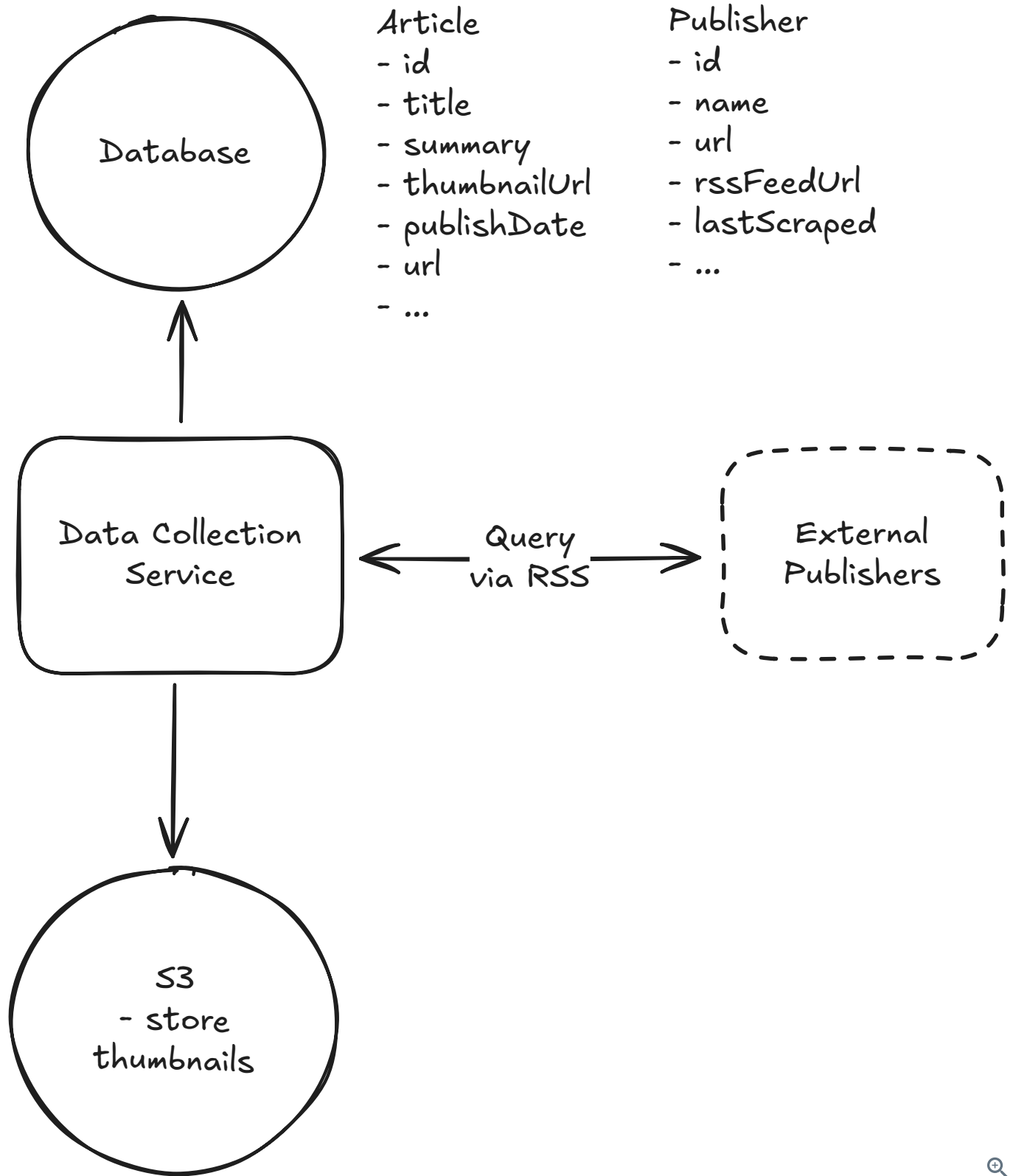
What is RSS? RSS is a simple XML format that allows publishers to syndicate their content to other websites and readers. It's a common format for news aggregators like Google News because it's a simple, standardized format that many publishers already support. RSS feeds are also relatively lightweight to parse, making them a good choice for our system.

RSS works over HTTP. We just need to make a GET request to the RSS feed URL to get the content. The response is an XML document that contains the article title, link, and other metadata.

You may be thinking, why not just point directly to the url of the source image hosted by the publisher rather than going through all the effort to download it and store it in our own Object Storage? This is a good question. The answer is that we want to be able to serve the images to users quickly and efficiently, and not rely on the publisher's servers which may be slow, overloaded, or go down entirely. Additionally, we want to be able to standardize the quality and size of the images to ensure a consistent user experience.

Now that we have data flowing in, we need to serve it to users. For this, we'll add a **Feed Service** that handles user requests:

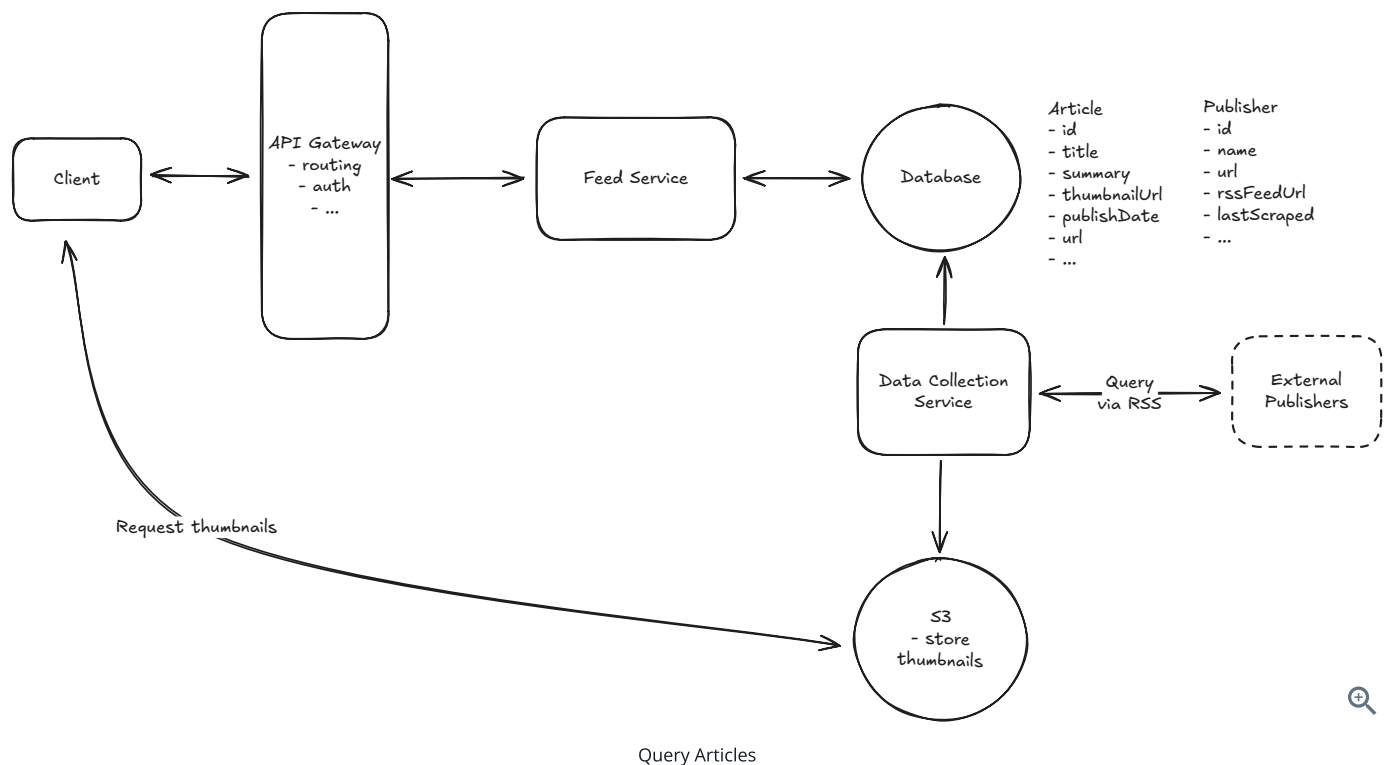




Data Collection

1. **Client:** Users interact with Google News through web browsers or mobile apps, requesting their personalized news feed
2. **API Gateway:** Routes incoming requests and handles authentication, rate limiting, and request validation before forwarding to appropriate services
3. **Feed Service:** Handles user feed requests by querying for relevant articles based on the user's region and formatting the response for consumption

We choose to separate the Feed Service from the Data Collection Service for several key reasons: they have completely different scaling requirements (read-heavy vs write-heavy), different update frequencies (real-time vs batch), and different operational needs (user-facing vs background processing).



When a user requests their news feed:

1. Client sends a GET request to `/feed?region=US&limit=20`
2. API Gateway routes the request to the Feed Service
3. Feed Service queries the Database for recent articles in the user's region, ordered by publish date
4. Database returns article data including metadata and media URLs pointing to Object Storage
5. Feed Service formats the response and returns it to the client via the API Gateway

2) Users should be able to scroll through the feed "infinitely"

Users expect to continuously scroll through their news feed without manual pagination. This requires implementing pagination that can handle loading new batches of content as users scroll.

Building on our existing architecture, we'll enhance the Feed Service to support simple offset-based pagination using page numbers and page sizes to fetch batches of articles.

When a user initially loads their feed:

1. Client sends GET request to `/feed?region=US&limit=20&page=1` (first page)
2. Feed Service queries for the first 20 articles in the user's region, ordered by publish date
3. Response includes articles plus pagination metadata (`total_pages` , `current_page`)
4. Client stores the current page number for the next request

As the user scrolls and approaches the end of current content:

1. Client automatically sends GET request to `/feed?region=US&limit=20&page=2`
2. Feed Service calculates the offset (`page-1 * limit`) and fetches the next 20 articles

3. Database query fetches articles with OFFSET and LIMIT clauses
4. Process repeats as user continues scrolling through pages

This provides a simple foundation for infinite scrolling, though it has some limitations around performance and consistency that we'll address in our deep dives.

3) Users should be able to click on articles and be redirected to the publisher's website to read the full content

This is easy - the browser handles it for us. When users click an article, the browser redirects to the article URL stored in our database, taking them directly to the publisher's website to read the full content.

Sites like Google News are aggregators, and they don't actually host the content themselves. They simply point to the publisher's website when a user clicks on an article.



In real Google News, they would track analytics on article clicks to understand user behavior and improve recommendations. We consider this out of scope, but here's how it would work: article links would point to Google's tracking endpoint like `GET /article/{article_id}` which logs the click event and returns a 302 redirect to the publisher's site. This click data helps train recommendation algorithms and measure engagement.

Ok, pretty straightforward so far. Let's layer on a little complexity with our deep dives.

Potential Deep Dives

At this point, we have a basic, functioning system that satisfies the core functional requirements of Google News - users can view aggregated news articles, scroll through feeds infinitely, and click through to publisher websites. However, our current design has significant limitations, particularly around pagination consistency and feed delivery performance at scale. Let's look back at our non-functional requirements and explore how we can improve our system to handle 100M DAU with low latency and global distribution.

1) How can we improve pagination consistency and efficiency?

Our current offset-based pagination approach has serious limitations when new articles are constantly being published. Consider a user browsing their news feed during a busy news day when articles are published every few minutes. With traditional page-based pagination, if a user is on page 2 and new articles get added to the top of the feed, the content shifts down and the user might see duplicate articles or miss content entirely when they request page 3. This creates a frustrating user experience where the same articles appear multiple times or important breaking news gets skipped.

With thousands of publishers worldwide publishing articles throughout the day, we might see 50-100 new articles per hour during peak news periods. A user spending just 10 minutes browsing their feed could easily encounter this pagination drift problem multiple times, seeing duplicate articles or missing new content that was published while they were reading.

So, what can we do instead?

Good Solution: Timestamp-Based Cursors



Great Solution: Composite Cursor with Article ID



Great Solution: Monotonically Increasing Article IDs



By implementing cursor-based pagination with monotonically increasing article IDs, we ensure consistent pagination that handles new content gracefully while maintaining the sub-200ms latency requirement for feed requests.

2) How do we achieve low latency (< 200ms) feed requests?

Our high-level design currently queries the database directly for each feed request, which creates significant performance bottlenecks at scale. With 100 million daily active users, each potentially refreshing their feed 5-10 times per day, we're looking at 500 million to 1 billion feed requests daily. Even with efficient indexing, querying millions of articles and filtering by region for each request could push response times well beyond our 200ms target.

Pattern: Scaling Reads

News aggregators like Google News showcase extreme **scaling reads** scenarios with billions of feed requests but relatively few article writes. This demands aggressive caching of regional feeds, and pre-computed article rankings. The key is that news consumption vastly outweighs news creation, making read optimization critical for sub-200ms response times.

Learn This Pattern

How can we make this more efficient?

Good Solution: Redis Cache with TTL



Great Solution: Real-time Cached Feeds with CDC



3) How do we ensure articles appear in feeds within 30 minutes of publication?

Our current approach of polling publisher RSS feeds every 3-6 hours creates a big problem: by the time we discover breaking news, users have already learned about it from social media, push notifications, or other news sources. In today's fast-paced news environment, a delay of several hours makes our news feed feel stale and irrelevant. When a major story breaks - whether it's a natural disaster, political development, or market-moving announcement - users expect to see it in their feeds within minutes, not hours.



Most of the time when this question is asked, especially when asked of mid-level or junior candidates, the interviewer will ask you to "black box" the ingestion pipeline. I choose to go over it here because it is not uncommon for more senior candidates to be asked how this would be implemented, at least at a high level.

Here's how we can dramatically reduce this discovery time.

Good Solution: Increased RSS Polling Frequency



Good Solution: Intelligent Web Scraping



Great Solution: Publisher Webhooks with Fallback Polling



This question is perfect for an informed back and forth with your interviewer. Start by asking them questions and building your way up. Can I black box the ingestion pipeline? If not, do our publishers maintain RSS feeds? Given we have such high traffic, can we assume publishers would be willing to implement webhooks to tell us when new articles are published?

By implementing a hybrid approach that combines frequent RSS polling for cooperative publishers, intelligent web scraping for sites without feeds, and webhooks for premium real-time partnerships, we can ensure that breaking news appears in user feeds within minutes rather than hours.

4) How do we handle media content (images/videos) efficiently?

Since we link users to publisher websites rather than hosting full articles, our media requirements are much simpler - we only need to display thumbnails in the news feed to make articles visually appealing and help users quickly identify content. However, with 100M+ daily users viewing feeds, even thumbnail delivery needs to be fast and cost-effective.

When we collect articles via RSS or scraping, we extract the primary image URL from each article and download a copy to generate better thumbnails. We need our own copies because publisher images can be slow to load, change URLs, or become unavailable, which would break our feed experience.

Let's analyze our options for thumbnail storage and delivery.

Bad Solution: Database Blob Storage



Good Solution: S3 Storage with Direct Links



Great Solution: S3 + CloudFront CDN with Multiple Sizes



By implementing S3 storage with CloudFront CDN distribution and multiple thumbnail sizes, we provide fast thumbnail loading globally while keeping storage costs minimal. Since users click through to publisher sites for full articles, we only need to improve the feed browsing experience with quick-loading, appropriately-sized thumbnails.

5) How do we handle traffic spikes during breaking news?

Breaking news events create massive traffic spikes that can overwhelm traditional scaling approaches. When major events occur - elections, natural disasters, or celebrity news - our normal traffic of 100M daily active users can spike to 10M concurrent users within minutes. During these events, everyone wants the latest updates simultaneously, creating a perfect storm of read traffic that can bring down unprepared systems.



Realistically, 10M concurrent users is a lot and probably an overestimate, but it makes the problem more interesting and many interviewers push you to design for such semi-unrealistic scenarios.

Fortunately, Google News has a natural advantage that makes scaling much more manageable than other systems: news consumption is inherently regional. Users primarily want fast access to local and national news from their geographic region. While some users do seek international news, the vast majority of traffic focuses on regional content - Americans want US news, Europeans want EU news, and so on.

This means we can deploy infrastructure close to users in each region, and each regional deployment only needs to handle the content and traffic for that specific area. Rather than building one massive global system, we can build several smaller regional systems that are much easier to scale and operate.

We'll still assume that each regional deployment needs to handle 10M concurrent users making feed requests. So let's evaluate each component in our design asking: what are the resource requirements at peak, does the current design satisfy the requirement, and if not, how can we scale the component to meet the new requirement?

Feed Service (Application Layer)

Our Feed Service needs to handle 10M concurrent users making feed requests. Even if each user only refreshes their feed once during a breaking news event, that's still 10M requests that need to be processed quickly. A single application server can typically handle 10,000 - 100,000 concurrent connections depending on the response complexity and hardware.

So one server, no matter how powerful, won't cut it.

The solution is horizontal scaling with auto-scaling groups. We deploy multiple instances of our Feed Service behind load balancers and use cloud auto-scaling to automatically provision new instances when CPU or memory utilization exceeds certain thresholds. With proper load balancing, we can distribute the 10M requests across dozens of application server instances, each handling a manageable portion of the traffic.

The key advantage is that Feed Services are stateless, making horizontal scaling straightforward. We can spin up new instances in seconds and tear them down when traffic subsides, paying only for resources during high-traffic periods.

Database Layer

Our database faces the most significant scaling challenge during traffic spikes. Even with efficient indexing, a single database instance cannot handle 10M concurrent read requests. The I/O subsystem, network bandwidth, and CPU resources all become bottlenecks that cannot be overcome through hardware upgrades alone.

Good news is we've already got our cache which should drastically reduce the load on our database. All read requests to fetch the feed should hit the cache, meaning our scale challenges are actually offloaded from the database to the cache.

Cache Layer (Redis)

Our Redis cache layer becomes critical during traffic spikes as it serves as the primary source for pre-computed regional feeds. With 10M users requesting feeds simultaneously, even our tuned cache queries could overwhelm a single Redis instance which can only serve ~100k requests per second.

The solution is read replicas. Each regional Redis master gets multiple read replicas to distribute the query load. Since we only have ~2,000 recent articles per region, each master can easily store all the regional content without complex sharding - the scaling challenge is purely about read throughput.



What if I'm not using Redis? No worries! The concept is the same. Use consistent hashing to shard the data across multiple instances and ensure each instance has a replica or two to handle the read load and failover.

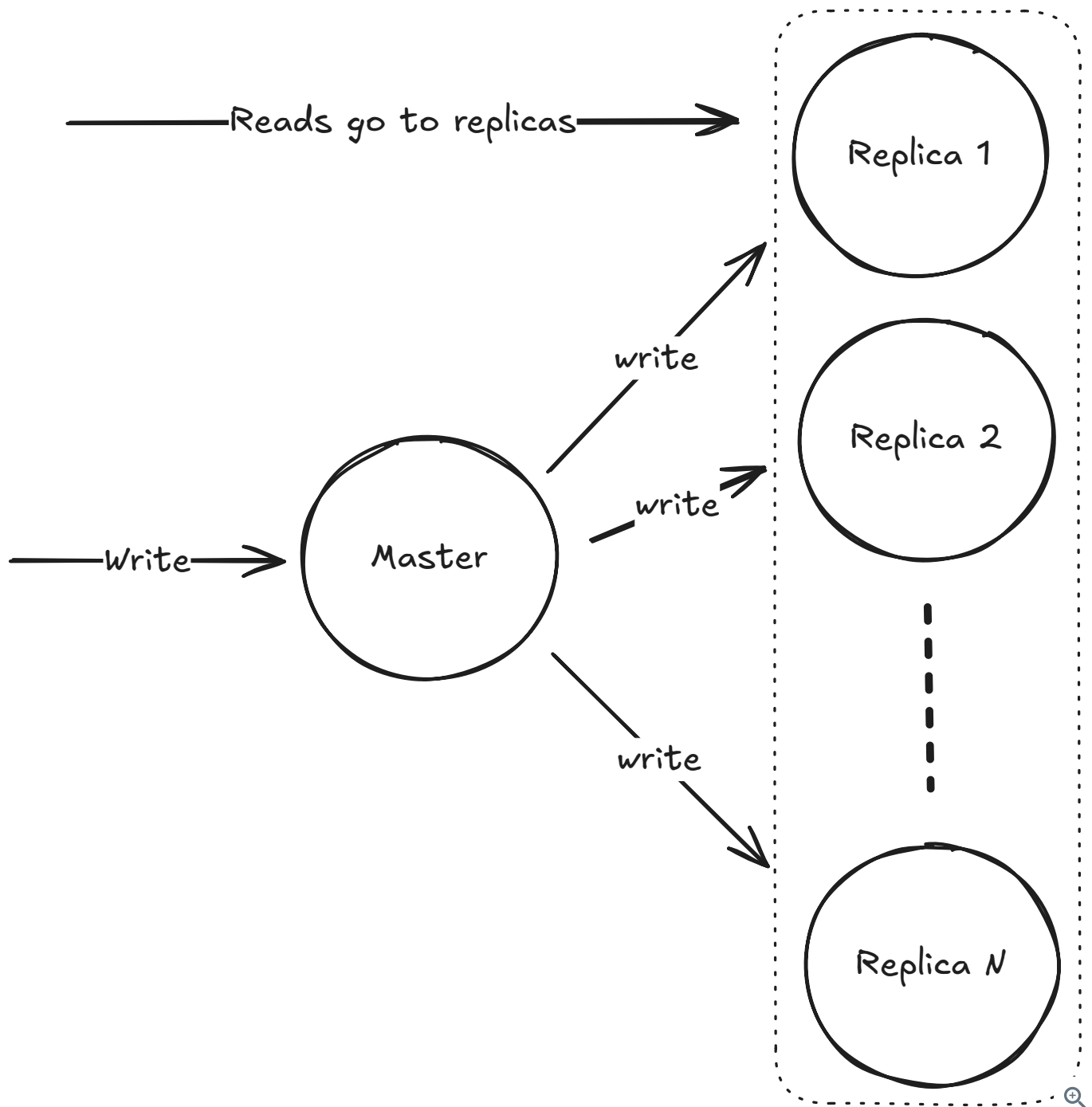
Let's work through the scaling math. With 10M concurrent users during traffic spikes and each Redis instance handling roughly 100k requests per second, we need 100 total Redis instances to handle the load.



Realistically, we don't need this many per region. Some regions are more popular than others, and we can scale up and down based on demand.

Setting this up is straightforward: write operations like new articles and cache updates go to the master, while read operations for feed requests are load-balanced across all replicas using round-robin or least-connections algorithms. With Redis Sentinel managing the cluster, if the master fails, one replica gets promoted to master automatically. The replication lag is typically under 200ms for Redis, which is perfectly acceptable for news feeds where users won't notice such small delays.





Redis Scaling

This handles our traffic spikes efficiently while keeping operational complexity manageable. During breaking news events, we can quickly spin up additional read replicas in the affected regions to handle increased load, then scale them back down when traffic normalizes.

This regional approach provides users with sub-50ms cache response times from their nearest cluster, traffic spikes in one region don't affect others, and we can scale each region independently based on local usage patterns. During breaking news events, the affected regions can add more read replicas while others remain at baseline capacity.

Bonus Deep Dives

Many users in the comments called out that when they were asked this question, they were asked about both categorization and personalization. I figured, given the interest, it was worth amending the breakdown to include

these topics.

6) How can we support category-based news feeds (Sports, Politics, Tech, etc.)?

Our current design only supports regional feeds like `feed:US` and `feed:UK`, but real news platforms organize content into categories like Sports, Politics, Technology, Business, and Entertainment. Users expect to browse specific topics rather than just getting a mixed regional feed.

Google News displays 25+ categories, each containing hundreds of daily articles. With 100M daily users, we might see up to 10M requests for specific categories during peak hours - Sports during game seasons, Politics during elections, or Tech during major product launches. Our current regional cache structure can't handle this granular filtering efficiently.

Consider what happens when a major sporting event occurs and 10M users simultaneously request Sports feeds. Our system would need to query the database for sports articles, filter results, and generate responses for each request. Even with regional caching, we'd be hitting the database millions of times for the same Sports content, creating performance bottlenecks.

Bad Solution: Database Query Filtering with Category Column



Good Solution: Pre-computed Category Feeds in Redis



Great Solution: In-memory filtering



7) How do we generate personalized feeds based on user reading behavior and preferences?

Our current system delivers the same regional feed to every user in a geographic area, but modern news platforms provide personalized experiences. Users expect feeds that prioritize topics they care about, publishers they trust, and content similar to articles they've previously engaged with.

The actual ranking/scoring function itself is usually a machine learning model, but we can abstract this away for our purposes. This isn't an MLE interview after all!

Bad Solution: Real-time Recommendation Scoring



Good Solution: Pre-computed User Feed Caches



Great Solution: Hybrid Personalization with Dynamic Feed Assembly



By implementing hybrid personalization with dynamic feed assembly, we deliver personalized news experiences that scale to 100M+ users while maintaining our sub-200ms response time requirements. The approach balances individual user interests with editorial importance and trending content, ensuring users get both relevant and globally significant news in their feeds.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

[✍ Start Quiz](#)

Mark as read



Add a comment...

B I </>      

Comment

☐ Anonymous

Posting as Slim Shady

Q Search comments

Sort By

Popular



Kamrul Alam

★ Top 5% • 4 months ago • edited 16 hours ago

How is this easy?

 45 [Reply](#)

R

RightBlackCaribou790

Premium • 7 days ago

Glad to see that i'm not alone in this thought!! The sheer possible combinations and permutations is endless. But then again i believe the onus is on you to satisfy the interviewer.

 0 [Reply](#)



Jerry

★ Top 5% • 5 months ago • edited 4 days ago

I think the requirement is a little misleading. "Regional feed" is very important based on the solution, used for geo-partitioning. However, the requirements doesn't mention region at all while the phase of "all over the world" implies it's a global feed.

 29 [Reply](#)

S

StrongCrimsonZebra608

★ Top 10% • 5 months ago

Highly agree.

 1 [Reply](#)

F

FavourablePlumBird366

★ Top 5% • 4 months ago • edited 1 day ago

Given the number of potential deep dives and the variety of solutions, this should be bumped to be a medium rather than an easy problem.

 23 [Reply](#)

E ExactAmethystLark466

★ Top 10% • 6 months ago • edited 1 day ago

Thanks for the writeup.

Just wanted to share that I was asked this question in an interview recently and the interviewer was mainly interested in this,

You have an API to call publishers which gives you only 25 results at a time. Publishers may also have their ratelimits. How will you ensure that you dont drop any news from the publisher. Some discussion around deduping as well around these requirements.

^ would require adaptive polling techniques I believe.

 15 [Reply](#)**U UnchangedBlackGorilla909****Premium** • 6 months ago • edited 23 days ago

^ would require adaptive polling techniques I believe.

could you elaborate more on this please? In my head if I were asked this question I would naively say "we would keep polling the api to adhere with the users rate limit so we would also have to make sure we're adhere with being "polite/god citizens" and have exponential back off retries".

Do you feel like this is a strong enough answer though, I can't imagine a world were we would have a rate limit less than 1 request per minute/5 minutes and they're publishing articles faster than that rate, I could be missing something and am curious what people think

 1 [Reply](#)**P PostgreEnjoyer**

• 6 months ago • edited 15 days ago

When using the Redis sorted set solution, if we use the timestamps as score, won't we encounter the same timestamp collision issue as mentioned in the "Timestamp-Based Cursor" section?

 3 [Reply](#)**Evan King**

Admin • 6 months ago

Redis maintains insertion order for elements with identical scores so should be fine

 3 [Reply](#)**S StrongCrimsonZebra608**

★ Top 10% • 5 months ago • edited 7 days ago

I think this should be mentioned in the article? Could be helpful

 8 [Reply](#)**V VerticalBlackFlea403****Premium** • 4 months ago • edited 26 days ago

based on <https://redis.io/docs/latest/develop/data-types/sorted-sets/>, when scores are the same, Redis sort members lexicographically by string

 1 [Reply](#)**H HandsomeIvoryCrow799**

★ Top 5% • 5 months ago • edited 5 months ago

If we're going to use [ZREVRANGEBYSCORE](#) then I think PostgreEnjoyer's question remains valid.

If there are **more than 10** articles sharing the same timestamp **1751180745**, we'll run into an infinite pagination loop querying the same 10 articles over and over again:

```
ZREVRANGEBYSCORE feed:US 1751180745 -inf LIMIT 10
```

I guess the article intended to use **ZRANGE** on the sorted set's element zero-based index instead (by not specifying BYSCORE argument)?

👍 0 [Reply](#)

F FancyTanGoldfish347

Premium • 5 months ago

I cannot find any documentation stating Redis will maintain insertion order for members with the same score. It specifically states that the members will be ordered lexicographically even with having the same score

👍 1 [Reply](#)

V VerticalBlackFlea403

Premium • 4 months ago

I think the score in Redis sorted set should be whatever used for cursor pagination. otherwise, it has the same issue as timestamp-based pagination

👍 0 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)

[Amazon SWE Interview Questions](#)

[Google SWE Interview Questions](#)

[Engineering Manager \(EM\) Interview Questions](#)

Learn

[Learn System Design](#)

[Learn DSA](#)

[Learn Behavioral](#)

[Learn ML System Design](#)

Guided Practice

Links

[FAQ](#)

[Pricing](#)

[Gift Mock Interviews](#)

[Gift Premium](#)

[Become a Coach](#)

[Our Coaches](#)

[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)

[Privacy Policy](#)

Contact

[About Us](#)

[Product Support](#)

7511 Greenwood Ave North

Unit #4238 Seattle

WA 98103

© 2025 Optick Labs Inc. All rights reserved.

